

Seals MES System

Git 版本管理与生产部署操作手册

版本: v2.0 | 日期: 2026年6月

仓库: github.com/Darren2676/rubber

第一篇: Git 版本管理策略

- 仓库现状与分支结构
- Git Flow 分支模型
- 版本号规范与 Tag 管理
- 提交信息规范 (Conventional Commits)
- 日常开发工作流
- 代码审查与合并流程

第二篇: 生产环境部署流程

- 环境准备清单
- 首次部署 (全新服务器)
- 部署验证

第三篇: 功能更新与Bug修复上线流程

- 新功能开发 → 生产上线全流程
- Bug 修复 (紧急热修复) 流程
- 版本发布操作步骤
- 升级后验证与回滚方案

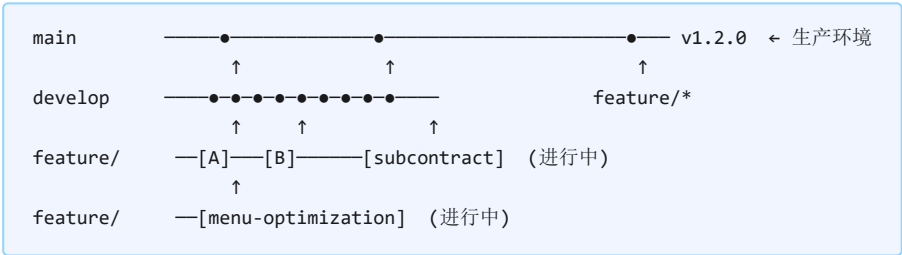
第一篇：Git 版本管理策略

1. 仓库现状与分支结构

1.1 当前仓库信息

配置项	当前值
远程仓库	https://github.com/Darren2676/rubber.git
主分支	main — 生产环境代码
开发分支	develop — 日常开发集成
功能分支	feature/production-subcontract 、 feature/menu-optimization
已发布 Tag	v1.0.0、v1.1.0、v1.2.0

1.2 当前分支实况



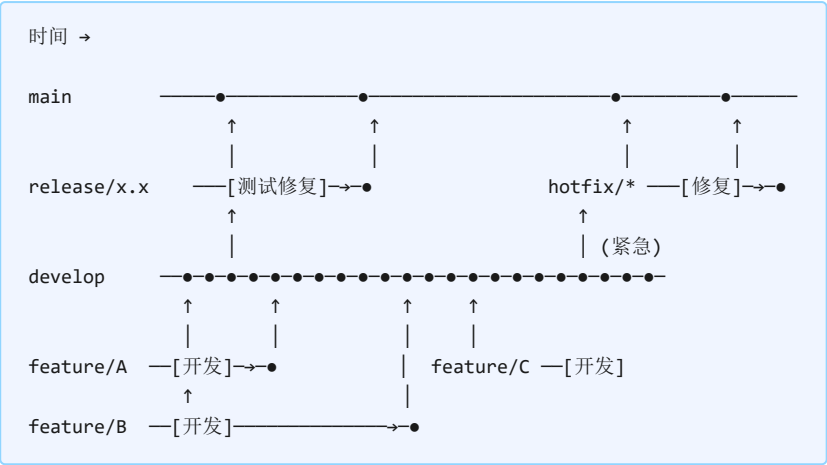
当前状态：**feature/production-subcontract** 是当前工作的功能分支（HEAD），**main** 处于 v1.2.0，下一次发布将为 v1.3.0。

2. Git Flow 分支模型

2.1 分支角色定义

分支	命名	来源	合并目标	生命周期	保护
main	固定	—	—	永久	锁定：禁止直接推送，仅通过 MR/PR 合并
develop	固定	main	—	永久	保护：仅通过 MR/PR 合并
feature/*	feature/功能名	develop	develop	合并后删除	无
release/*	release/x.x.x	develop	main + develop	合并后删除	保护：仅修复 Bug
hotfix/*	hotfix/问题描述	main	main + develop	合并后删除	无

2.2 完整分支流程图



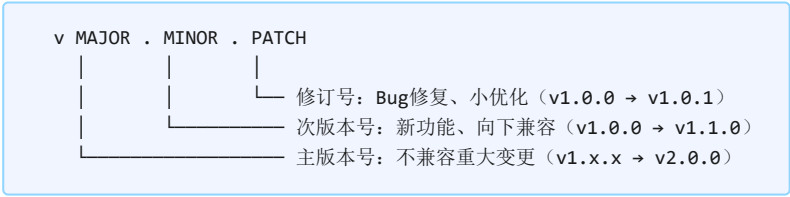
2.3 GitHub 分支保护设置

在 GitHub 仓库 → Settings → Branches 中配置：

分支	保护规则
main	需要 Pull Request 审查 (≥1人)、禁止直接推送、要求 CI 通过、不允许强制推送
develop	需要 Pull Request 审查 (≥1人)、禁止直接推送、不允许强制推送
release/*	禁止直接推送 (仅允许 Bug 修复的小型提交)

3. 版本号规范与 Tag 管理

3.1 语义化版本 (SemVer 2.0)



3.2 版本号同步位置

位置	文件	操作
Git Tag	—	<code>git tag -a v1.3.0 -m "v1.3.0 - 生产委外功能"</code>
后端	<code>server/package.json</code>	修改 "version": "1.3.0"
前端	<code>client/package.json</code>	修改 "version": "1.3.0"

3.3 Tag 操作命令

```
# 创建 Tag
git tag -a v1.3.0 -m "v1.3.0 - 新增生产委外功能"

# 查看所有 Tag
git tag -l

# 推送 Tag 到远程
git push origin v1.3.0

# 推送所有 Tag
git push origin --tags

# 删除本地 Tag（打错了需要重打）
git tag -d v1.3.0

# 删除远程 Tag
git push origin --delete v1.3.0
```

4. 提交信息规范 (Conventional Commits)

类型	说明	示例
feat	新功能	feat(production): 新增委外发料和收回功能
fix	Bug 修复	fix(warehouse): 修复入库单撤回时库存未恢复
docs	文档更新	docs: 更新部署手册中IIS配置部分
style	样式调整	style(ui): 调整表格列宽和对齐方式
refactor	代码重构	refactor(auth): 抽取JWT验证为独立中间件
perf	性能优化	perf(db): 优化物料查询SQL避免全表扫描
test	测试相关	test(e2e): 添加生产入库流程端到端测试
chore	构建/工具	chore(deps): 升级Vite到7.4.0
db	数据库变更	db(migration): 新增委外发料表

提交前检查：提交信息用中文或英文均可，但同一功能分支内应保持一致。类型（type）必须使用英文小写。

5. 日常开发 workflow

5.1 开始新功能开发

1

从 develop 拉取最新代码

```
git checkout develop
git pull origin develop
```

2 创建功能分支

```
git checkout -b feature/委外发料
```

命名规则: **feature/功能简述**, 使用英文或拼音, 小写加连字符。

3 开发过程中频繁提交

```
# 前端开发
cd client && npm run dev
# 修改代码...
git add src/components/OutsourcingIssue.vue
git commit -m "feat(production): 添加委外发料单页面组件"

# 后端开发
cd server && npm run dev
# 修改代码...
git add src/modules/production/outsourcingIssue/
git commit -m "feat(production): 实现委外发料API接口"
```

4 推送功能分支到远程

```
git push -u origin feature/委外发料
```

定期推送确保代码有远程备份, 也便于其他开发者查看进度。

5 本地测试通过后, 创建 Pull Request

- 在 GitHub 上发起 **feature/委外发料** → **develop** 的 Pull Request
- 填写 PR 描述: 变更内容、测试情况、影响范围
- 指派审查人 (Reviewer)

6 代码审查通过后合并

```
# 审查人审核通过后, 在 GitHub 上点击 "Merge pull request"
# 或本地合并:
git checkout develop
git pull origin develop
git merge feature/委外发料
git push origin develop
```

7 删除功能分支

```
# 本地删除
git branch -d feature/委外发料

# 远程删除
git push origin --delete feature/委外发料
```

5.2 同步 develop 最新代码到功能分支

如果功能分支开发周期较长，需要定期同步 develop 的最新代码避免冲突：

```
# 在功能分支上执行
git checkout feature/委外发料
git merge develop
# 解决冲突后提交
git push origin feature/委外发料
```

6. 代码审查与合并流程

6.1 Pull Request 检查清单

#	检查项
1	代码是否遵循项目编码规范
2	是否包含无用的 console.log 或调试代码
3	是否有硬编码的密码、密钥或敏感信息
4	数据库变更是否编写了迁移脚本（up + down）
5	新增 API 是否有输入校验
6	是否影响现有功能（回归风险评估）
7	前端构建是否通过（npm run build 无错误）
8	后端编译是否通过（npm run build 无错误）

6.2 合并冲突解决

产生冲突时的操作步骤：

```
# 1. 在功能分支上合并 develop
git checkout feature/xxx
git merge develop

# 2. Git 提示冲突后，手动编辑冲突文件
# <<<<<< HEAD      ← 你的代码
# =====          ← 分隔线
# >>>>>> develop   ← develop 的代码

# 3. 解决所有冲突后
git add .
git commit -m "merge: 解决与develop的合并冲突"

# 4. 推送
git push origin feature/xxx
```

第二篇：生产环境部署流程

7. 环境准备清单

7.1 服务器要求

组件	要求	验证命令
操作系统	Windows Server 2016+ 或 Windows 10/11	<code>winver</code>
Node.js	v20.x LTS	<code>node -v</code>
SQL Server	2008 R2+ (推荐 2014 SP3)	SSMS 连接测试
IIS	10.0 + URL Rewrite 2.1 + ARR 3.0	<code>iisreset /status</code>
PM2 (可选)	latest	<code>pm2 -v</code>
Git	2.x+	<code>git --version</code>

7.2 部署前置检查

- ☐ SQL Server 已安装且 TCP/IP 协议已启用 (端口 1433)
- ☐ 数据库 XYMES / PowderMom 已创建
- ☐ 防火墙已开放端口：80 (IIS)、3000 (后端)、1433 (数据库-如需远程)
- ☐ Node.js 已安装 (`node -v` 确认)
- ☐ IIS 已安装 + URL Rewrite 模块 + ARR 模块
- ☐ ARR 已启用代理 (Server Proxy Settings → Enable proxy)
- ☐ 目录 `D:\SealsMES\` 已创建

8. 首次部署（全新服务器）

1 克隆代码仓库

```
cd /d D:\
git clone https://github.com/Darren2676/rubber.git SealsMES
cd SealsMES
git checkout main
```

如果服务器无法访问外网，可在开发机上下载 ZIP 包后通过 U 盘拷贝。

2 安装后端依赖并编译

```
cd /d D:\SealsMES\server
npm install --production
npm run build
```

若服务器无网络，在开发机执行 **npm install** 后连同 **node_modules** 一起拷贝。

3 配置环境变量

```
# 创建 server/.env 文件，内容：
PORT=3000
DB_HOST=127.0.0.1
DB_PORT=1433
DB_NAME=PowderMom
DB_USER=sa
DB_PASSWORD=cowork
JWT_SECRET=请替换为一个随机的长字符串（32位以上）
JWT_EXPIRES_IN=24h
JWT_REFRESH_EXPIRES_IN=7d
UPLOAD_DIR=uploads
MAX_FILE_SIZE=10485760
```

重要：.env 文件包含数据库密码，已在 .gitignore 中排除，不会被提交到 Git。生产环境务必修改 JWT_SECRET 为强随机字符串。

4 创建上传目录

```
mkdir D:\SealsMES\server\uploads
mkdir D:\SealsMES\server\logs
```

5 测试后端启动

```
cd /d D:\SealsMES\server
node dist/server.js
```

看到以下输出表示成功：

```
数据库连接成功
数据库同步完成
服务器运行在端口 3000
API地址：http://localhost:3000/api
```

按 **Ctrl+C** 停止测试。

6 配置 PM2 管理后端进程

```
# 安装 PM2
npm install -g pm2

# 使用已有的 PM2 配置
cd /d D:\SealsMES\server
pm2 start ecosystem.config.js

# 设置开机自启
npm install -g pm2-windows-startup
pm2-startup install
pm2 save
```

常用 PM2 命令：

```
pm2 list      # 查看进程状态
pm2 logs      # 查看日志
pm2 restart all # 重启所有
pm2 stop all   # 停止所有
```

7 构建并部署前端

在开发机上执行构建，将产物复制到服务器：

```
# 在开发机上
cd /d "d:\powder\Seals MES System\client"
npm install
npm run build
# 将 dist/ 文件夹复制到服务器 D:\SealsMES\client\dist\
```

8 配置 IIS 网站

创建 `D:\SealsMES\client\dist\web.config`:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="API Proxy" stopProcessing="true">
          <match url="^api/(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:3000/api/{R:1}" />
        </rule>
        <rule name="Uploads Proxy" stopProcessing="true">
          <match url="^uploads/(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:3000/uploads/{R:1}" />
        </rule>
        <rule name="Vue Router">
          <match url=".*" />
          <conditions>
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

然后在 IIS 管理器中 → 右键"网站" → "添加网站":

配置项	值
网站名称	SealsMES
物理路径	D:\SealsMES\client\dist
端口	80

9. 部署验证

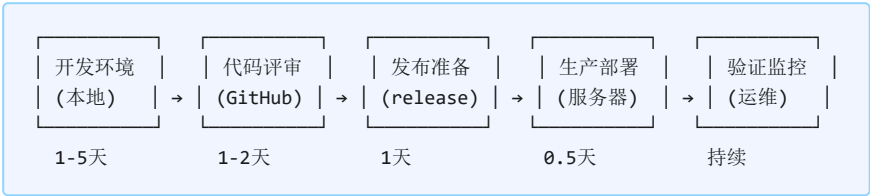
#	检查项	操作	预期结果
1	后端 API	浏览器访问 http://localhost:3000/api	返回 JSON { "success": true }
2	前端页面	浏览器访问 http://localhost	显示登录页面
3	用户登录	输入 admin / admin123	进入系统仪表板
4	数据库连接	浏览器访问 http://localhost:3000/api/v1/health	"database": "ok"
5	基础功能	依次点击物料管理、客户管理等菜单	正常显示数据
6	外网访问	客户端浏览器访问 http://服务器IP	显示登录页面

第三篇：功能更新与 Bug 修复上线流程

10. 新功能开发 → 生产上线全流程

以下以 "新增委外发料功能" 为例，展示从开发到上线的完整流程。

10.1 全流程概览



10.2 阶段一：开发环境（本地开发）

A1 从 develop 创建功能分支

```
git checkout develop
git pull origin develop
git checkout -b feature/委外发料
```

A2 本地开发（前端 + 后端同时启动）

```
# 终端1: 启动后端（已有 .env 指向本地开发数据库）
cd server
npm run dev
# → 后端运行在 http://localhost:3000

# 终端2: 启动前端（Vite 开发服务器）
cd client
npm run dev
# → 前端运行在 http://localhost:5173
```

开发过程中前端自动代理 API 到后端，支持热更新。

A3 频繁提交代码

```
# 每个小功能点提交一次
git add .
git commit -m "feat(production): 添加委外发料单页面组件"
git push origin feature/委外发料
```

A4 本地自测

- ☐ 前端构建通过: `cd client && npm run build` 无报错
- ☐ 后端编译通过: `cd server && npm run build` 无报错
- ☐ 新功能 CRUD 操作正常
- ☐ 旧功能不受影响 (回归测试)
- ☐ 数据库迁移脚本已编写且可正向执行和回滚

10.3 阶段二: 代码评审 (Pull Request)

B1 发起 Pull Request

- 在 GitHub 仓库中点击 "Pull requests" → "New pull request"
- base: `develop`, compare: `feature/委外发料`
- 填写 PR 标题和描述 (变更内容、测试结果、影响范围)
- 在右侧面板指定 Reviewer

B2 Reviewer 审查代码

审查者检查: 代码规范、逻辑正确性、安全性、数据库迁移、前后端一致性。

如有问题, 在代码行上添加 Comment, 开发者修改后推送新提交, PR 自动更新。

B3 审查通过 → 合并到 develop

```
# GitHub 上点击 "Merge pull request" (选择 "Squash and merge" 推荐)
# 或本地执行:
git checkout develop
git pull origin develop
git merge feature/委外发料
git push origin develop
git branch -d feature/委外发料
```

10.4 阶段三: 发布准备 (Release 分支)

C1 创建 release 分支

```
git checkout develop
git pull origin develop
git checkout -b release/1.3.0
```

C2 更新版本号

```
# 编辑 server/package.json → "version": "1.3.0"
# 编辑 client/package.json → "version": "1.3.0"
git add .
git commit -m "chore: bump version to 1.3.0"
```

C3 在测试环境（或本地）最终验证

- ☐ 全量构建通过（前端 + 后端）
- ☐ 核心业务流程正常（登录→创建单据→审批→入库→出库）
- ☐ 新功能完整可用
- ☐ 数据库迁移可正常执行
- ☐ 无控制台错误或异常日志

C4 合并到 main 并打 Tag

```
git checkout main
git pull origin main
git merge release/1.3.0
git tag -a v1.3.0 -m "v1.3.0 - 新增委外发料功能和Bug修复"

# 同步合并回 develop
git checkout develop
git merge release/1.3.0

# 推送
git push origin main
git push origin develop
git push origin v1.3.0

# 删除 release 分支
git branch -d release/1.3.0
git push origin --delete release/1.3.0
```

10.5 阶段四：生产环境部署

部署前务必执行以下备份操作！

D1 备份数据库

```
-- 在 SSMS 中执行
BACKUP DATABASE PowderMom
TO DISK = 'D:\Backup\PowderMom_v1.2.0_20260601.bak'
WITH FORMAT, COMPRESSION;
GO
```

D2 备份现有应用

```
xcopy D:\SealsMES D:\Backup\SealsMES_v1.2.0_20260601 /E /I /Y
```

D3 停止后端服务

```
# PM2 方式
pm2 stop seals-mes-api

# 或 Windows 服务方式
net stop SealsMES-API
```

D4 拉取最新代码

```
cd /d D:\SealsMES
git fetch origin
git checkout main
git pull origin main
```

确认当前 Tag: **git describe --tags** 应输出 **v1.3.0**

D5 更新后端

```
cd /d D:\SealsMES\server
npm install --production
npm run build
```

D6 执行数据库迁移（如有）

```
# 手动迁移脚本模式（当前项目使用方式）
# 启动后端时 initDatabase() 会自动执行迁移检查

# 或使用 Sequelize CLI（如果已配置）
npx sequelize-cli db:migrate
```

注意：当前项目在 `server/src/models/index.ts` 中使用手动 SQL 迁移方式，启动时自动检测并执行。如需新增迁移，在该文件中添加 SQL 检查语句。

D7 构建前端并部署

```
# 在开发机或服务器上
cd /d D:\SealsMES\client
npm install
npm run build

# 复制 dist 到 IIS 目录（如果构建在开发机）
xcopy dist D:\SealsMES\client\dist /E /I /Y
```

D8 启动后端服务

```
# PM2 方式
cd /d D:\SealsMES\server
pm2 start seals-mes-api

# 或 Windows 服务方式
net start SealsMES-API

# 查看启动日志确认正常
pm2 logs seals-mes-api --lines 20
```

D9 验证部署

#	检查项	操作	通过标准
1	服务状态	pm2 list	status: online
2	数据库	查看启动日志	"数据库连接成功"
3	API	访问 /api/v1/health	"database":"ok"
4	登录	浏览器登录系统	正常进入仪表板
5	新功能	操作新功能模块	CRUD 正常
6	旧功能	抽查核心页面	数据正常

11. Bug 修复（紧急热修复）流程

11.1 适用场景

生产环境发现紧急 Bug（如：登录失败、入库数据错误、审批流程异常），需要立即修复并上线。

11.2 热修复流程

步骤	操作	分支	时间
1	从 main 创建 hotfix 分支	hotfix/fix-login	即时
2	在 hotfix 分支修复 Bug	hotfix/fix-login	1-4h
3	本地测试通过	hotfix/fix-login	0.5h
4	合并到 main + 打补丁 Tag	main → v1.3.1	即时
5	部署到生产环境	服务器	0.5h
6	同步合并回 develop	develop	即时

11.3 具体操作步骤

H1 从 main 创建热修复分支

```
git checkout main
git pull origin main
git checkout -b hotfix/fix-login-token
```

命名：**hotfix/问题简述**

H2 修复 Bug 并提交

```
# 修复代码...
git add .
git commit -m "fix(auth): 修复登录Token验证过期判断错误
原因: JWT过期时间比较逻辑使用了本地时间而非服务器时间
影响: 用户登录后短时间内Token被判过期
修复: 使用iat+exp字段计算, 而非Date.now()比较"
git push origin hotfix/fix-login-token
```

提交信息务必写清楚：Bug 原因、影响范围、修复方法。

H3 合并到 main 并打补丁版本 Tag

```
git checkout main
git pull origin main
git merge hotfix/fix-login-token
git tag -a v1.2.1 -m "v1.2.1 - 修复登录Token验证异常"
git push origin main v1.2.1
```

H4 执行生产部署（精简版）

```
# 1. 备份（至少备份修改涉及的文件）
# 2. 停止服务
pm2 stop seals-mes-api

# 3. 拉取代码
cd /d D:\SealsMES
git pull origin main

# 4. 重新编译后端
cd server && npm run build

# 5. 重新构建前端（如前端有改动）
cd ../client && npm run build

# 6. 启动服务
cd ../server && pm2 start seals-mes-api

# 7. 验证修复
# 打开浏览器，执行之前报错的操作，确认 Bug 已修复
```

H5 同步合并回 develop

```
git checkout develop
git pull origin develop
git merge hotfix/fix-login-token
git push origin develop

# 删除 hotfix 分支
git branch -d hotfix/fix-login-token
git push origin --delete hotfix/fix-login-token
```

关键：务必同步合并回 develop，否则下次从 develop 发版时这些修复会丢失！

12. 升级后验证与回滚方案

12.1 升级后完整验证清单

#	验证项	方法	通过标准
1	PM2 进程状态	<code>pm2 list</code>	status: online, uptime 持续增长
2	后端启动日志	<code>pm2 logs --lines 50</code>	无 ERROR, "数据库连接成功"
3	健康检查	<code>curl http://localhost:3000/api/v1/health</code>	healthy, db: ok
4	登录功能	浏览器登录 admin/admin123	成功进入仪表板
5	基础数据 CRUD	新增/编辑/删除一条客户记录	各操作正常
6	单据流程	创建→审批 一个生产单	状态流转正确
7	报表查询	打开出入库报表页面	数据正常展示
8	导入导出	导入Excel + 导出Excel	文件生成正常, 数据正确
9	新功能	操作本次新增的功能	功能正常
10	移动端	用手机浏览器访问	页面响应式正常

12.2 回滚方案

如果升级后发现问题无法快速修复，立即执行回滚：

R1 停止当前服务

```
pm2 stop seals-mes-api
```

R2 恢复应用代码到上一个版本

```
# 方式A: Git 回退
cd /d D:\SealsMES
git checkout v1.2.0

# 方式B: 还原备份
xcopy D:\Backup\SealsMES_v1.2.0 D:\SealsMES /E /I /Y
```

R3 恢复数据库（如有表结构变更）

```
-- 在 SSMS 中执行
USE master;
GO
ALTER DATABASE PowderMom SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO
RESTORE DATABASE PowderMom
FROM DISK = 'D:\Backup\PowderMom_v1.2.0_20260601.bak'
WITH REPLACE;
GO
ALTER DATABASE PowderMom SET MULTI_USER;
GO
```

R4 重新编译并启动

```
cd /d D:\SealsMES\server
npm run build
pm2 start seals-mes-api
```

R5 验证回滚成功 + 通知相关人员

```
# 执行 12.1 的验证清单
# 通知使用方系统已回滚，升级暂时推迟
```

12.3 快速参考卡

场景	操作	命令
创建功能分支	从 develop 创建	<code>git checkout -b feature/xxx develop</code>
合并到 develop	GitHub PR 或本地 merge	<code>git checkout develop && git merge feature/xxx</code>
发布新版本	release → main + tag	<code>git tag -a v1.x.0 && git push origin v1.x.0</code>
紧急修复	hotfix → main + tag + develop	<code>git checkout -b hotfix/xxx main</code>
部署到生产	拉取 → 编译 → 重启	<code>git pull && npm run build && pm2 restart</code>
回滚	切 Tag → 编译 → 重启	<code>git checkout v1.x.0 && npm run build && pm2 restart</code>
查看当前 Tag	确认版本	<code>git describe --tags</code>
重启服务	应用更新	<code>pm2 restart seals-mes-api</code>